

3D DATA PROCESSING - LAB 3

Point Cloud Registration - Report

Step 1: Descriptor

The goal of this stage was to find a coarse **initial alignment** between the source and target clouds without any prior pose information. Since ICP is a local method and can only refine an already-close alignment, a reliable global estimate is essential before any refinement is attempted.

- Both clouds were first downsampled using a voxel size set to 2% of the bounding box diagonal. This makes the **voxel size scale-aware** across all three datasets without needing to hardcode a value for each one. Surface normals were then estimated on the downsampled clouds using a hybrid **KD-tree search** with a **radius of 2x the voxel size**, which captures enough local neighbourhood to produce stable normal directions.
- **FPFH (Fast Point Feature Histogram) descriptors** were computed for each point using a **radius of 5x the voxel size**. FPFH encodes the local surface geometry as a 33-dimensional histogram, and the larger radius ensures it captures enough context to distinguish features from one another. Correspondences were then found using `RegistrationRANSACBasedOnFeatureMatching`, with two correspondence checkers applied to prune false matches: an `EdgeLength` checker (ratio 0.9) that rejects pairs whose geometric structure is inconsistent, and a `Distance` checker that enforces a maximum allowable distance of 1.5x the voxel size. The **RANSAC convergence criterion** was set to 10^7 iterations at confidence 0.999 to ensure a stable result before passing the transformation to the ICP stage.

Step 2: ICP Refinement

Once the descriptor step produced an initial alignment, ICP was used to refine it by iteratively **minimizing the distance between corresponding points** in the two clouds.

At each iteration, `find_closest_point` uses `KDTreeFlann` to find the nearest neighbour in the target cloud for each source point. Correspondences where the distance exceeds the threshold are discarded as **outliers**. The threshold itself is set adaptively as ***min (2 x current RMSE, 5% of the bounding box diagonal)***, which scales to the actual misalignment rather than relying on a fixed value. Convergence is checked after each iteration: if the **change in inlier RMSE falls below $1e-4$** , the loop terminates early.

- **SVD-based transformation**
The SVD-based transformation is the **closed-form solution** to the point-to-point registration problem. The centroids of both matched point sets are computed and subtracted, producing zero-mean matrices P (source) and Q (target). The cross-covariance matrix $W = P^T Q$ is then decomposed using `Eigen::JacobiSVD`, giving $W = U S V^T$. The optimal rotation is $R = V U^T$. A **reflection check** is applied: if $\det(R) < 0$, the sign of the last column of V is flipped before recomputing R , which handles degenerate configurations where the SVD would otherwise return an improper rotation. The translation is then $t = \text{tgt_centroid} - R * \text{src_centroid}$.

- **LM-based transformation**

The LM-based transformation solves the same problem non-linearly using the **Ceres Solver**. A templated `PointDistance` functor is defined for each correspondence, computing the residual as the **3D distance between the rotated source point and its target match**. The rotation is parameterized as a 3D axis-angle vector `[rx, ry, rz]`, and `ceres::AngleAxisRotatePoint` applies it inside the functor, enabling automatic differentiation. Each residual block uses `ceres::HuberLoss(1.0)` as a robust loss function, which reduces the influence of outlier correspondences that survive the distance threshold check by down-weighting large residuals. The full 6-DOF parameter vector `[rx, ry, rz, tx, ty, tz]` is initialised to zero at each ICP iteration and optimized by Ceres. After solving, the axis-angle vector is converted to a rotation matrix via `Eigen::AngleAxisd` before assembling the final 4x4 transform.

Problems Encountered

A subtle but important issue was found in how the incremental ICP transformations were composed. At each iteration, the ICP loop computes a small delta transform `T_delta` that aligns the source in its current (already transformed) position (in the code it is simply named `transformation`, without underscore). The accumulated transformation must therefore be updated as `transformation_ = T_delta * transformation_`, not the reverse. The wrong order (`transformation_ * T_delta`) would compose the transforms in the wrong order, producing an accumulated matrix that does not represent the true accumulated pose. This was identified because the ICP inlier RMSE converged correctly each iteration, but `compute_rmse()` (which reconstructs the cloud from the original source using the accumulated matrix) reported a much higher value, revealing the inconsistency.

Note: This issue was independently identified and resolved prior to its discussion on the course forum and is documented here as part of the implementation record.

Results – Step 3: Methods Analysis

All five ICP variants were tested on three datasets (Bunny, Dragon, Vase) under perturbed initial conditions. The descriptor-estimated pose was deliberately corrupted by adding random rotation and translation noise to simulate realistic misalignment. **Two noise levels are reported:** 0°/0mm (no perturbation, baseline performance) and 10°/5mm (maximum perturbation, robustness under stress).

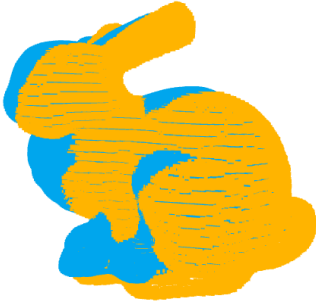
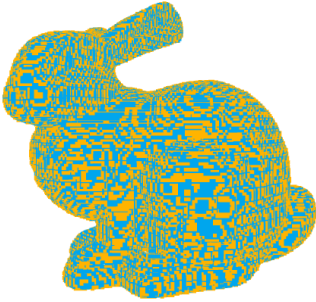
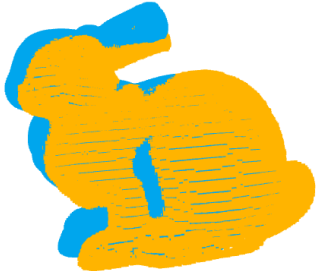
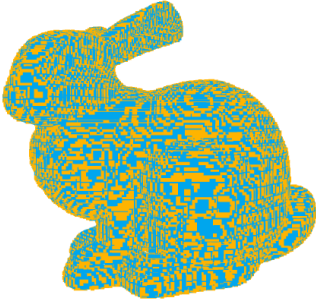
Iteration counts for Open3D methods were read directly from the debug output (ICP Iteration #x lines) since the API reports 0. All runs used the adaptive threshold ***min (2 x current RMSE, 5% of bounding box diagonal)*** and convergence criterion ***relative_rmse = 1e-4***.

The screenshots show the point clouds under 10°/5mm perturbation: the **noisy initial state (left)** and the final registered result **after ICP refinement (right)**. Source cloud is shown in orange, target in blue.

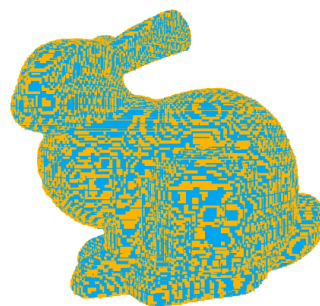
The `ply_files` folder contains the merged registered point clouds for all three datasets across all five methods, produced with no perturbation (0°/0mm noise).

Bunny dataset

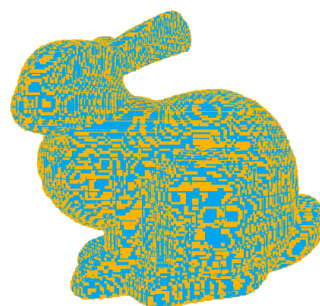
Rot [deg]	Trans [mm]	Method	Iterations	Time [ms]	Final RMSE	Initial RMSE
0	0	svd	9	688.0	0.2124	0.3796
0	0	lm	6	10840.6	0.2124	0.4006
0	0	o3d-p2point	10	52.3	0.2124	0.4195
0	0	o3d-p2plane	3	24.8	0.2124	0.3873
0	0	o3d-gen	3	88.1	0.2124	0.4881
10	5	svd	24	1836.8	0.2124	3.1499
10	5	lm	23	82429.0	0.2124	3.1457
10	5	o3d-p2point	22	123.7	0.2124	2.8266
10	5	o3d-p2plane	9	85.3	0.2124	4.6287
10	5	o3d-gen	5	109.4	0.2124	3.4449

Method	Noisy Initial State	After Refinement
svd		
lm		

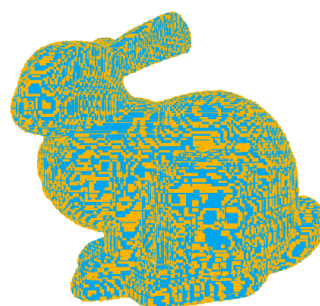
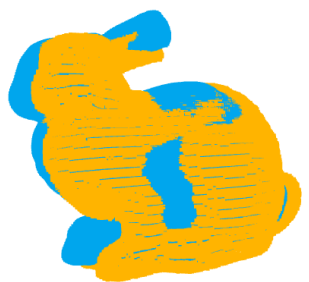
o3d-p2point



o3d-p2plane



o3d-gen



Dragon dataset

Rot [deg]	Trans [mm]	Method	Iterations	Time [ms]	Final RMSE	Initial RMSE
0	0	svd	8	230.7	0.4329	0.5219
0	0	lm	15	11284.8	0.4329	0.7182
0	0	o3d-p2point	12	47.0	0.4329	0.5431
0	0	o3d-p2plane	4	22.5	0.4329	0.6962
0	0	o3d-gen	3	52.3	0.4329	0.6014
10	5	svd	20	648.6	0.4329	3.5128
10	5	lm	32	39389.2	0.4329	3.1435
10	5	o3d-p2point	24	81.9	0.4329	4.1749
10	5	o3d-p2plane	8	42.4	0.4329	3.7876
10	5	o3d-gen	7	73.9	0.4329	3.9645

Method	Noisy Initial State	After Refinement
svd		
lm		

o3d-p2point



o3d-p2plane

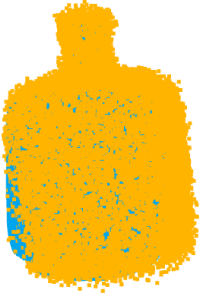
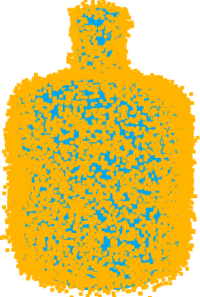
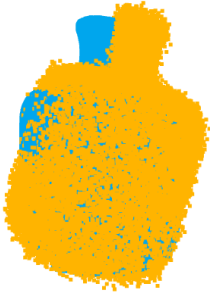
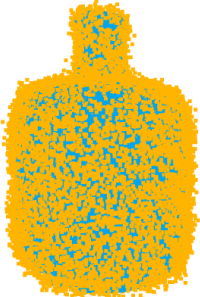


o3d-gen

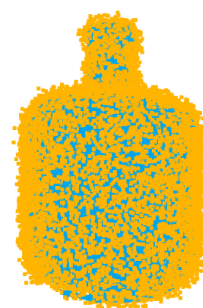
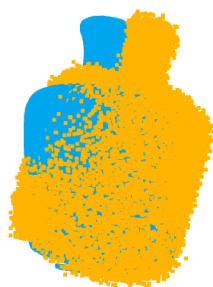


Vase dataset

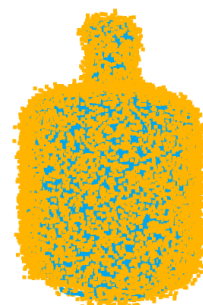
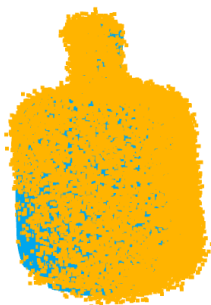
Rot [deg]	Trans [mm]	Method	Iterations	Time [ms]	Final RMSE	Initial RMSE
0	0	svd	19	1704.9	1.2600	1.4540
0	0	lm	21	68197.7	1.1700	1.3507
0	0	o3d-p2point	29	179.3	1.2303	1.6118
0	0	o3d-p2plane	10	77.0	1.1682	1.4978
0	0	o3d-gen	21	288.0	1.1718	1.3506
10	5	svd	22	2119.0	1.4165	3.5909
10	5	lm	64	204935.0	1.2332	3.3197
10	5	o3d-p2point	35	227.9	1.1752	3.5385
10	5	o3d-p2plane	5	33.0	1.1682	2.4286
10	5	o3d-gen	18	291.3	1.1714	3.6635

Method	Noisy Initial State	After Refinement
svd		
lm		

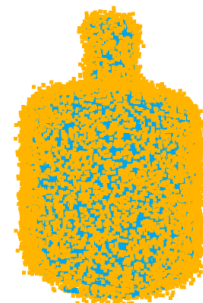
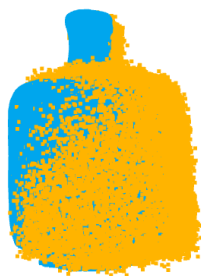
o3d-p2point



o3d-p2plane



o3d-gen



Discussion

SVD vs LM: bunny and dragon

SVD and LM always converge to the same final RMSE on the bunny and dragon, which is expected since both minimize the same point-to-point objective. The difference is purely speed: SVD is a closed-form solve per iteration and completes in under 2 seconds even at 10°/5mm noise, while LM runs an internal iterative optimization via Ceres on top of the outer ICP loop, reaching 82 seconds on the bunny and 39 seconds on the dragon for the same result.

SVD vs LM: vase

On the vase the picture changes. At 0°/0mm, LM achieves a lower final RMSE than SVD (1.170 vs 1.260), and at 10°/5mm LM again outperforms SVD (1.233 vs 1.417). This is the one case where the two methods diverge in accuracy. The reason is likely that the vase's symmetry causes the inlier correspondence set to be noisier than on the other datasets.

Open3D methods: bunny and dragon

On the bunny and dragon, all five methods converge to the same final RMSE regardless of noise level. The differences are in speed and iterations. Point-to-plane is consistently the fastest: 3 iterations on the bunny at 0°/0mm, 4 on the dragon, converging in well under 50ms. Point-to-point requires more iterations to reach the same RMSE (10 on bunny, 12 on dragon at 0°/0mm). Generalized ICP matches point-to-plane in final RMSE and iterations on the dragon (3 vs 3) but is always slower.

Open3D methods: vase

On the vase, no method fully resolves the alignment. Point-to-plane achieves the lowest final RMSE at both noise levels (1.168 at 0°/0mm, 1.168 at 10°/5mm) and does so in the fewest iterations (10 and 5 respectively), which reflects the advantage of normal-based minimization on smooth surfaces. Point-to-point requires 29 iterations at 0°/0mm and still reaches a higher RMSE (1.230). Generalized ICP settles at 1.172 but runs for 21 iterations at 0°/0mm, more than point-to-plane for a marginally worse result.

Vase dataset and descriptor fitness

The RANSAC fitness for the vase is consistently around 0.92-0.97, noticeably below the bunny and dragon which both achieve fitness 1.0. This reflects the near-cylindrical symmetry of the vase: FPFH descriptors struggle to produce unambiguous correspondences on rotationally symmetric surfaces. As a result, ICP inherits a weaker initial alignment, and all methods converge to a higher RMSE floor of ~1.17-1.42.

Qualitative results

Despite the quantitative differences in final RMSE across datasets and methods, the qualitative results are consistently good. The registered point clouds for all three datasets show well-aligned surfaces with no visible gross misalignment, even at 10°/5mm perturbation. The vase, while settling at a higher RMSE floor, still produces a visually coherent registration.